StratosEvent

Enterprise Event Management & Registration Platform

INDUSTRY PROJECT REPORT

A Project Report submitted in partial fulfilment of the requirements
of the
TCS iON Industry Project Programme
in
Web Technology — Full Stack Development

Industry Project Title	StratosEvent — Enterprise Event Management and Registration Platform
Name of Candidate	Abhinav Sankar
Name of the Institute	LEAD College (Autonomous), Palakkad
Platform	TCS iON Digital Learning Hub
Documentation Date	November 2026
Start Date	August 2026
End Date	November 2026
Total Effort (hrs.)	Approx. 120 hrs.
Project Environment	Web Browser / Local Development Server
Tools Used	Node.js, Express.js, MySQL, HTML5, CSS3, JavaScript, Bootstrap 5, VS Code, Postman, GitHub

Submitted by Abhinav Sankar • November 2026

Acknowledgements

I would like to express my sincere gratitude to TCS iON for providing the Industry Project Programme as a structured bridge between academic learning and real-world software engineering practice. The programme's milestone-driven format, documentation standards, and emphasis on professional delivery shaped both the technical scope and the quality of this project.

I am deeply thankful to the faculty and coordinators at LEAD College (Autonomous), Palakkad, for their continuous guidance, feedback, and encouragement. Their willingness to review architectural decisions, testing evidence, and interface design helped raise the project from a classroom exercise to a submission-ready industry artefact.

I would also like to acknowledge the open-source communities behind Node.js, Express.js, MySQL, Bootstrap, and the wider JavaScript ecosystem. Their well-maintained documentation and libraries made it possible to build a secure, full-stack platform within the project timeline.

Finally, I am grateful for the opportunity to work on a problem with genuine practical value — making event discovery, registration, moderation, and attendance verification reliable for organisers and attendees alike. That sense of purpose guided every architectural, security, and design decision recorded in this report.

Table of Contents

1. Abstract....4
2. Introduction....4
3. Problem Statement....5
4. Objectives....5
5. Scope of the Project....6
6. System Overview....7
7. Approach / Methodology — Tools and Technologies Used....15
8. System Architecture....16
9. Database Design....18
10. Module-wise System Description....19
11. Booking and Ticketing Logic....21
12. User Interface Design....22
13. Key Features....26
14. Admin Panel....26
15. Challenges and Opportunities....27
16. Advantages of the System....28
17. Limitations....29
18. Testing Documentation....29
19. Reflections on the Project....31
20. Recommendations....32
21. Outcome / Conclusion....33
22. Enhancement Scope....33
23. Link to Code and Executable File....34
24. Research Questions and Responses....34
25. References....35

1. Abstract

StratosEvent is a full-stack, enterprise-grade event management and registration platform built to solve a problem that most organisers still handle with disconnected spreadsheets, unvalidated web forms, and paper sign-in sheets: how do you publish an event, fill its seats fairly, issue trustworthy passes, and know exactly who attended?

The application follows a decoupled three-tier architecture. The presentation layer is a responsive, dependency-light single-page interface built with HTML5, CSS3, modern ES6+ JavaScript, and Bootstrap 5. The application layer is a RESTful API built with Node.js and Express.js. The data layer is MySQL 8.0 using the InnoDB storage engine, with foreign keys, check constraints, composite unique indexes, and transactional row locking. Every action visible in the interface — registration, booking, approval, check-in, analytics — is backed by a defined API endpoint and a database operation.

The platform supports three distinct access levels: public visitors who browse and search the published catalogue, authenticated participants who reserve tiered passes and manage their own tickets, and administrators who control the full event lifecycle, moderate registrations, operate a gate check-in scanner, and export executive analytics. Seat reservation is protected by an atomic transaction using SELECT ... FOR UPDATE, while duplicate bookings and duplicate check-ins are blocked by database-level unique constraints. A public-registration hardening rule unconditionally forces the participant role, mitigating privilege-escalation attacks (CWE-269).

StratosEvent is also visually deliberate. Its Hawkins National Laboratory / Stranger Things retro-futuristic theme — void black, laboratory crimson, upside-down teal, amber CRT indicators, monospace instrumentation, and responsive glow effects — gives the platform a memorable identity without compromising usability. The finished system was verified with 35 automated backend tests and 26 end-to-end operational flows, both achieving a 100% pass rate, with zero horizontal overflow at mobile width and complete preservation of the seeded database baseline.

The complete source code for this project is available at: https://github.com/abhinavsankar27-bot/StratosEvent

2. Introduction

Campus symposiums, enterprise workshops, technical hackathons, webinars, and conferences all share the same operational core: a fixed venue capacity, a set of eligible attendees, a registration window, a moment of admission, and a need to report what happened. When those steps are handled by separate tools, the gaps between them become failure points — seats are oversold, participants register twice, passes are copied, and attendance figures are reconstructed manually after the event.

This project was developed as part of the TCS iON Industry Project Programme and places the developer in the role of a full-stack engineer tasked with building a production-quality event platform from first principles. The aim was not to produce a static prototype or a frontend-only demonstration, but a genuine client–server system in which authentication, authorisation, data integrity, and reporting are implemented at the correct layer of the stack.

The project deliberately avoids a heavy client-side framework. The frontend is served as static assets and uses vanilla JavaScript with a small set of focused modules, while Bootstrap 5 supplies the responsive grid and modal behaviour. This zero-bundler approach keeps first load fast, eliminates

build-chain fragility, and makes the codebase easy to inspect during assessment. The backend uses Express middleware for authentication and role checks, parameterised SQL for every query, and MySQL transactions for the booking critical path.

The visual identity is drawn from the Hawkins National Laboratory aesthetic of Stranger Things: a dark laboratory palette, warning-red accents, CRT scanlines, monospace status readouts, and restrained neon glow. The theme is not decoration for its own sake — it creates clear hierarchy, distinguishes administrative surfaces from public ones, and makes system state (live, published, checked-in, sold out) immediately legible. The screenshots in Section 6 show the finished application running against the live MySQL database.

3. Problem Statement

The central problem is that event registration is a concurrency-sensitive, identity-sensitive workflow, yet it is often implemented as a simple form that writes a row to a table. That simplification produces predictable failures at exactly the moments that matter: when demand is high, when a ticket is presented at the gate, and when an organiser needs to reconcile the final attendee list.

Stakeholder analysis identified the following specific limitations in conventional approaches:

- Simultaneous registrations for the last available seats cause overbooking because availability is read and then updated in separate steps without a lock.
- Participants register more than once for the same event, either accidentally or deliberately, wasting capacity and distorting demand forecasts.
- Public registration forms that trust a client-supplied role field allow privilege escalation, giving ordinary users administrative control.
- Ticket codes are sequential or guessable, making pass sharing and attendance fraud difficult to detect at the gate.
- Attendance is recorded on paper or in a separate spreadsheet, so the online roster and the physical gate list cannot be reconciled reliably.
- Organisers lack real-time visibility into registration velocity, capacity utilisation, check-in ratios, and participant demographics.
- Heavy single-page frameworks introduce large dependency trees, slow first paint, and fragile build pipelines for what is fundamentally a forms-and-tables application.

Each of these failures has a technical root cause: missing transactions, missing constraints, missing authorisation checks, or missing observability. StratosEvent addresses those root causes directly at the database and API layers, then exposes the results through an interface that makes the state of every event and every pass understandable at a glance.

4. Objectives

The project defined ten measurable objectives, all of which were implemented and verified:

1. Design and deliver a complete full-stack application using Node.js, Express.js, MySQL 8.0, and a responsive browser client, demonstrating end-to-end integration across presentation, application, and data tiers.
2. Implement role-based access control for public visitors, participants, and administrators, with clearly separated permissions enforced on the server rather than only in the interface.

3. Provide a live event catalogue with keyword search, category filtering, computed seat availability, and detailed event views including venue, schedule, capacity, and ticket tiers.
4. Implement a concurrency-safe reservation engine that serialises competing bookings with row-level locking and guarantees that event capacity is never exceeded.
5. Prevent duplicate registrations and duplicate gate check-ins through composite unique constraints and explicit conflict responses at the API layer.
6. Generate a cryptographically unique, traceable ticket code for every confirmed registration and present it in a participant ticket wallet with status and tier information.
7. Give administrators a unified control console for event creation, lifecycle management, registration approval or rejection, and attendee roster inspection.
8. Implement a gate check-in workflow that verifies a pass before admission, records the check-in time and status, and rejects a second scan of the same pass.
9. Deliver an executive analytics engine covering event counts, registration status breakdown, attendance confirmation ratio, capacity utilisation, participant demographics, and CSV export.
10. Harden authentication with bcrypt password hashing, HS256-pinned JSON Web Tokens, production secret guards, parameterised SQL, and CWE-269 mitigation; then verify the system with a comprehensive automated and end-to-end test suite.

5. Scope of the Project

5.1 What This Project Covers

- A responsive public catalogue that loads published events from the API and renders them as accessible event cards with category badges, schedule, venue, capacity, and live seat indicators.
- Instant client-side keyword search across title, description, and venue, combined with a category filter for workshops, webinars, conferences, hackathons, and seminars.
- Participant registration and login with bcrypt-hashed credentials and signed JWT sessions; public registration always provisions the participant role.
- A tiered pass model supporting General, VIP, Student, and Early Bird allocations, including per-event tier quotas stored in the database and enforced during booking.
- Atomic seat reservation inside a MySQL transaction using SELECT ... FOR UPDATE, with capacity checks, duplicate detection, and rollback on failure.
- Unique ticket generation, a participant ticket wallet, and self-service cancellation that immediately releases the reserved seat back to the availability pool.
- A full administrator console covering registration approvals, gate check-in and ticket inspection, event creation with tier allocation, event lifecycle changes, and real-time analytics.
- Attendance verification with check-in timestamps, status values (checked_in, late, absent), gate notes, and duplicate check-in rejection.
- An executive reporting engine with KPI cards, event-wise capacity and attendance tables, participant domain distribution, and an authorised CSV roster export.
- A distinctive, fully responsive Hawkins Laboratory interface verified at desktop, tablet, and mobile widths with zero horizontal overflow.

- A complete documentation set comprising a Software Requirements Specification, requirements traceability matrix, system flow diagrams, wireframes, API reference, test log, deployment checklist, technical manual, and this report.

5.2 What This Project Does Not Cover

- Real payment processing. Ticket tiers are operational categories; no monetary transaction or payment gateway is integrated.
- Automated email or SMS notifications. Confirmation and approval feedback is delivered in the interface only.
- Camera-based QR scanning. The gate scanner accepts a ticket code typed or pasted into the verification field; a camera decoder is not included.
- Cloud deployment and container orchestration. The system is designed to run on a local development machine or a single application server.
- A native Android or iOS application. The responsive web interface adapts to mobile browsers.
- Multi-tenant organisation isolation, single sign-on, and calendar synchronisation. These are documented as enhancement scope, not delivered functionality.

6. System Overview

StratosEvent is a three-tier web application. The browser client renders the interface and manages view state; the Express API enforces authentication, authorisation, validation, and business rules; MySQL stores users, events, registrations, and attendance with referential integrity. All communication uses HTTP and JSON, and all protected requests carry a Bearer token in the Authorization header.

The screenshots in this section were captured from the running application at http://localhost:3000, connected to the live Express API at http://localhost:5000 and the seeded MySQL database. They show real data, not mock placeholders.

6.1 Landing Page and Event Discovery

The landing page establishes the platform identity immediately. The navigation bar carries the STRATOSEVENT wordmark, a Hawkins ’83 badge, links to Explore Events and My Tickets, and separate Login and Admin Portal actions. The hero presents the headline “DISCOVER EVENTS WORTH ATTENDING” above a concise value proposition and two primary calls to action. A value strip below reinforces the four promises of the platform: digital event passes, tiered pass options, secure seat reservation, and rapid gate check-in.

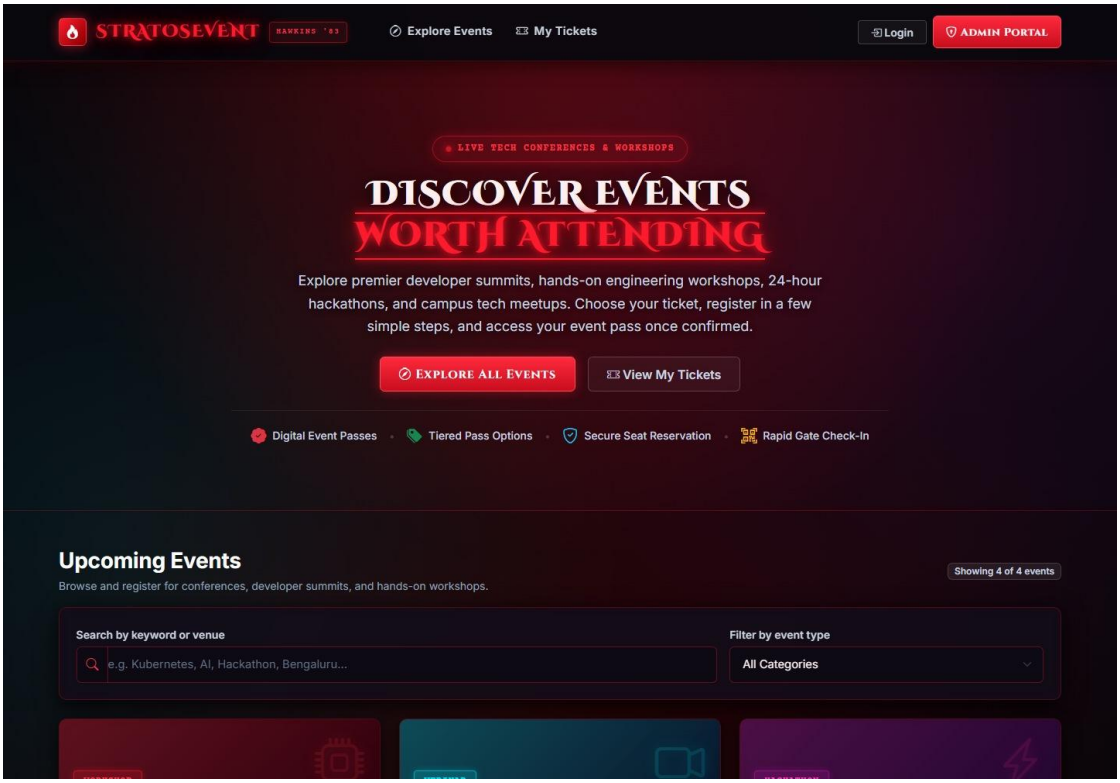


Figure 1: Landing page — Hawkins Laboratory hero, navigation, and live event teaser

Scrolling into the catalogue reveals the Upcoming Events section, a live result counter, a keyword search field, and a category filter. Each event card presents its type, title, description, formatted date range, venue, and a capacity indicator. The published catalogue shown in Figure 2 contains four live events — a Kubernetes workshop, an AI and LLM webinar, a 24-hour hackathon, and a zero-trust security seminar — drawn directly from the events table; the fifth seeded event remains in draft and is therefore absent from the public view.

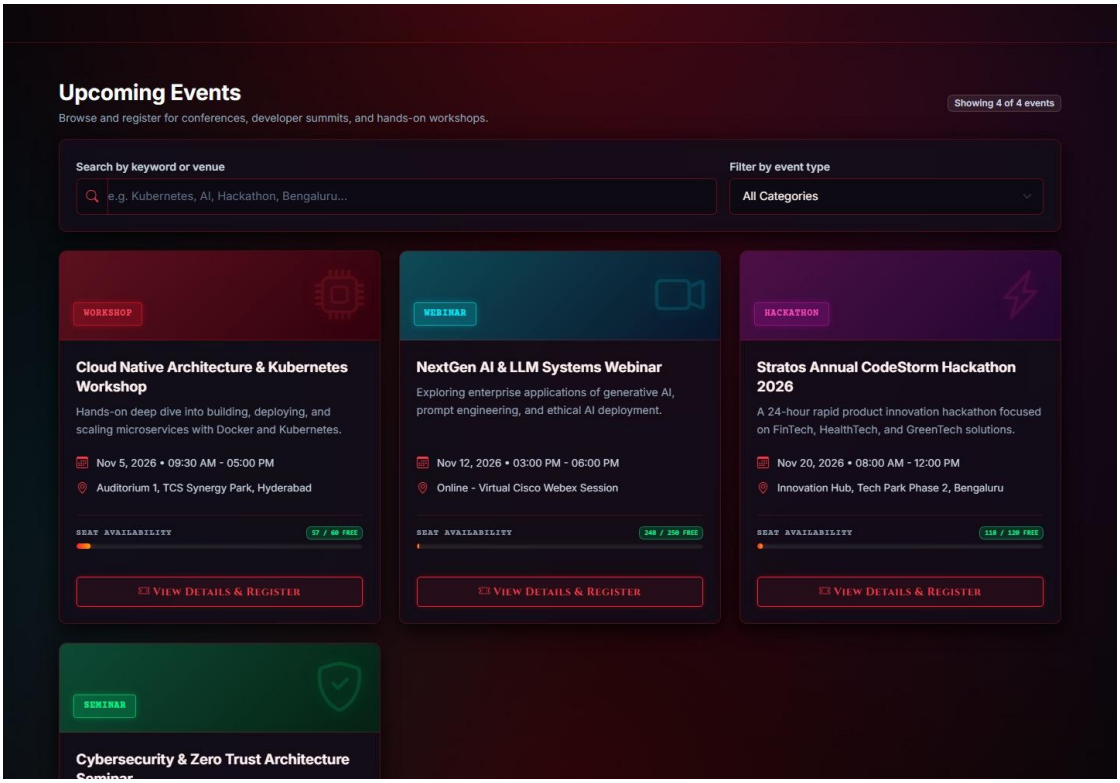


Figure 2: Event catalogue — live search, category filter, and computed seat availability

6.2 Authentication and Registration

Authentication is handled in a focused modal rather than a separate page, so a participant never loses the event they were viewing. The modal provides Sign In and Create Account tabs, loads demo credentials for evaluation, and displays the role associated with the selected account. Passwords are submitted over JSON to the login endpoint and compared against the stored bcrypt hash; the returned token is held in localStorage and attached to every protected request.

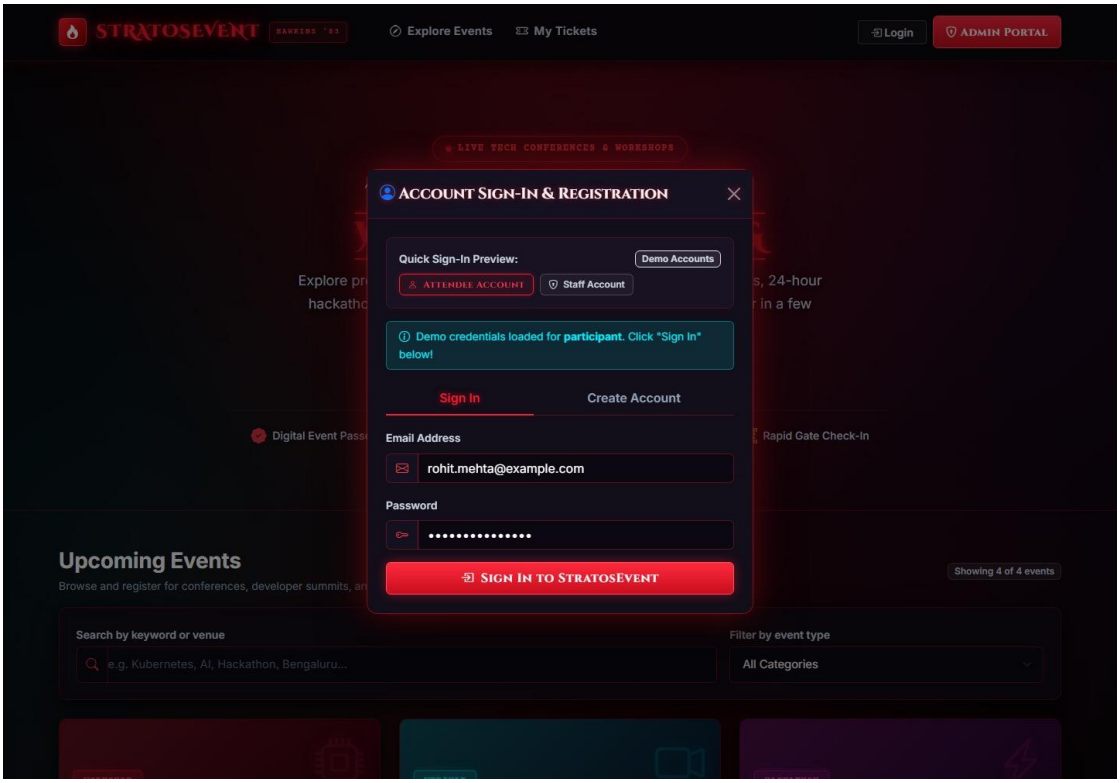


Figure 3: Account sign-in and registration modal with role-aware demo access

6.3 Event Details and Tier Selection

Selecting View Details opens the event detail modal. It repeats the essential decision information — description, category, schedule, venue, capacity, and organiser — and then presents the pass-tier selector. Availability is computed per tier from the event's tier capacity and the number of active registrations in that tier. The register action is disabled when the event is full or the selected tier has no remaining seats, giving the user immediate feedback before a request is sent.

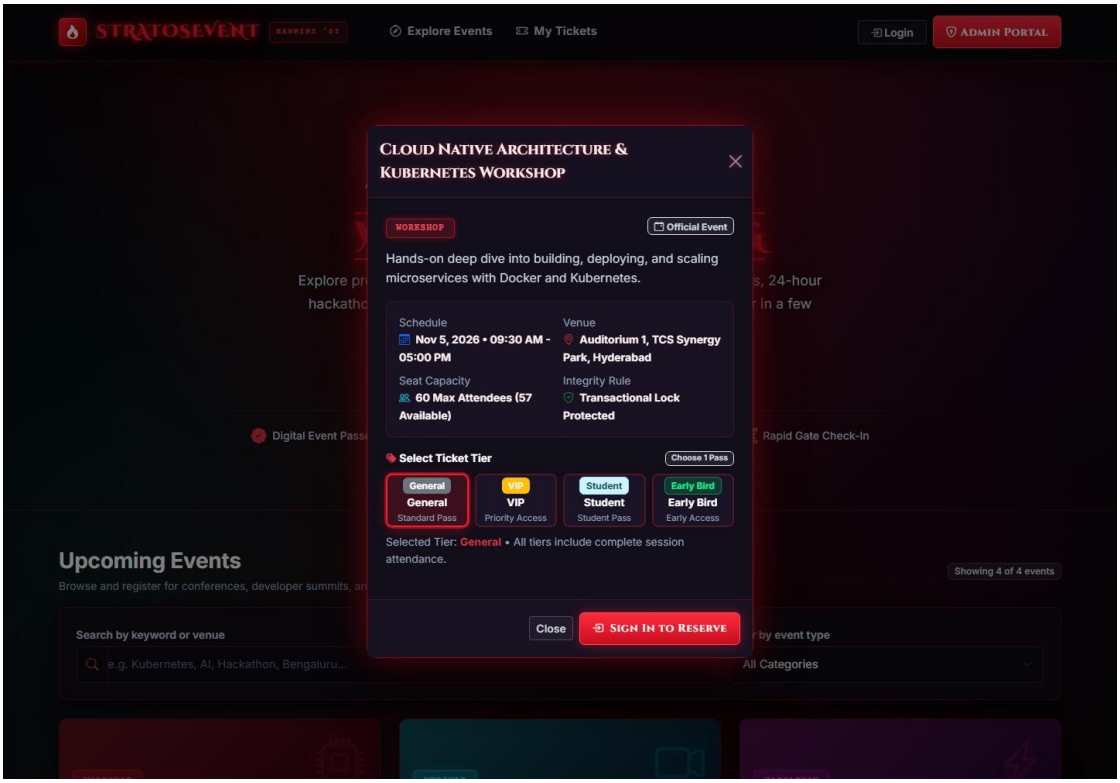


Figure 4: Event detail modal — schedule, venue, live capacity, and tier selection

6.4 Participant Ticket Wallet

The My Tickets view is the participant's record of every pass they hold. It lists the event, venue, date, ticket code, tier, and status, and provides a cancellation action for active bookings. A cancelled pass releases its seat immediately and is marked cancelled rather than deleted, preserving the audit trail. Figure 5 shows the wallet for a seeded participant account, including checked-in and approved passes.

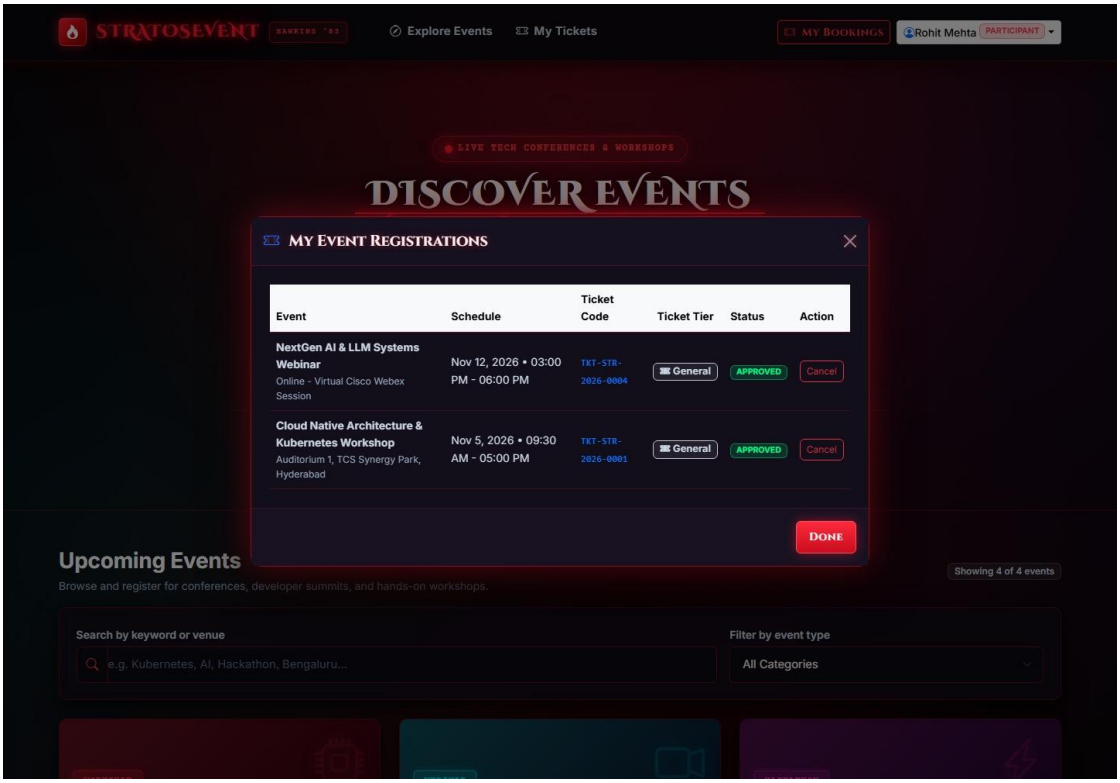


Figure 5: Participant ticket wallet — pass codes, tier, status, and cancellation controls

6.5 Administrative Control Console

The administrator console is a single modal with four operational tabs. Registration Approvals filters the roster by event and ticket tier and exposes approve or reject actions. Gate Check-In & Scanner verifies a pass before admission and records the outcome. Create & Manage Events publishes a new event and allocates capacity across tiers. Analytics & Reports presents live KPIs, per-event performance, demographics, and a CSV export. Figures 6 to 9 show each tab operating against the live database.

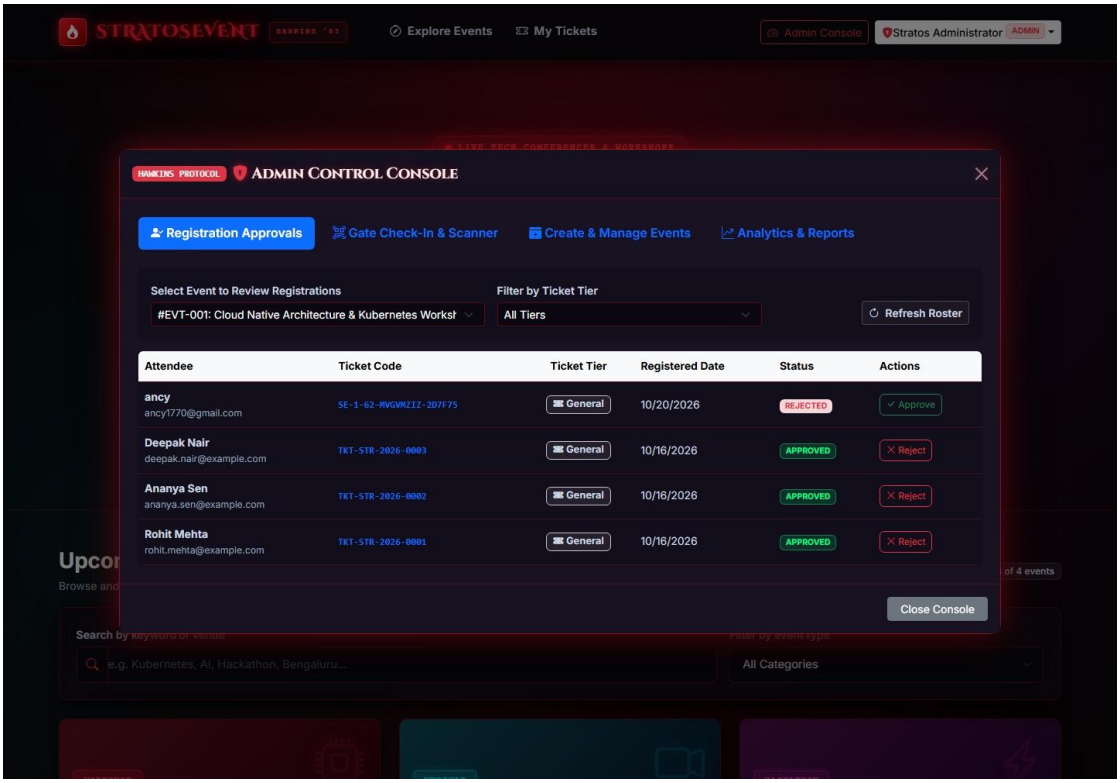


Figure 6: Registration approvals — attendee roster with ticket codes, tiers, and moderation actions

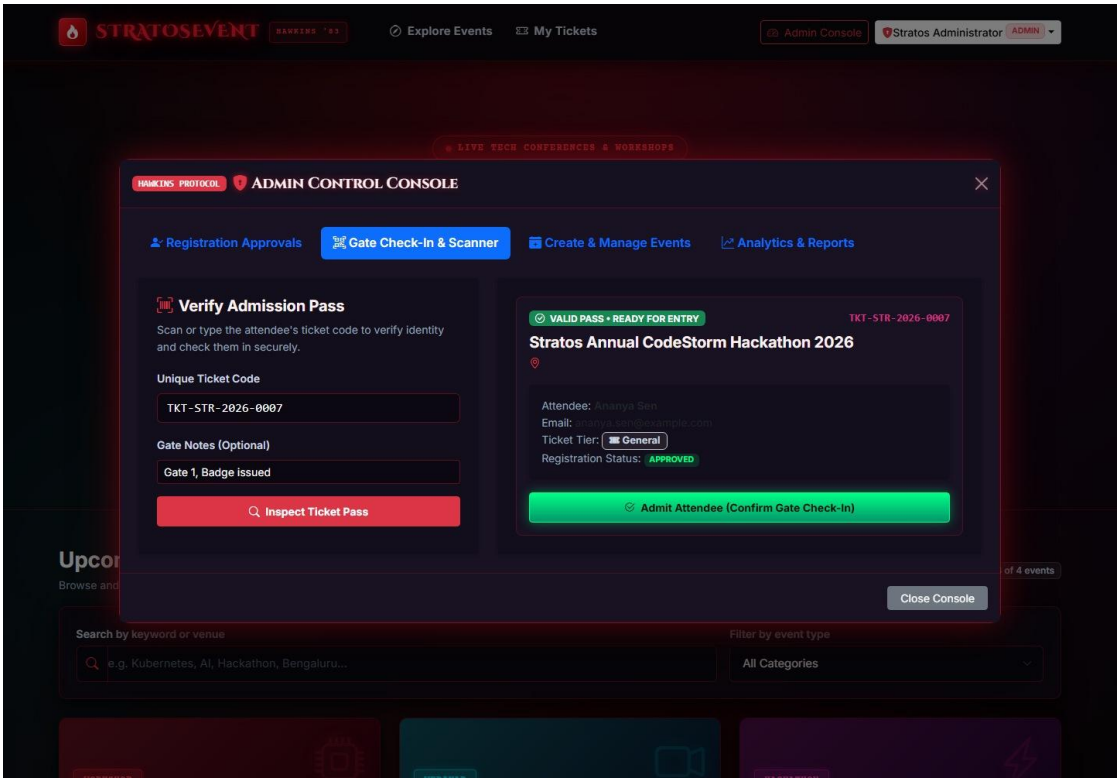


Figure 7: Gate check-in and scanner — pass verification with attendee identity and admission controls

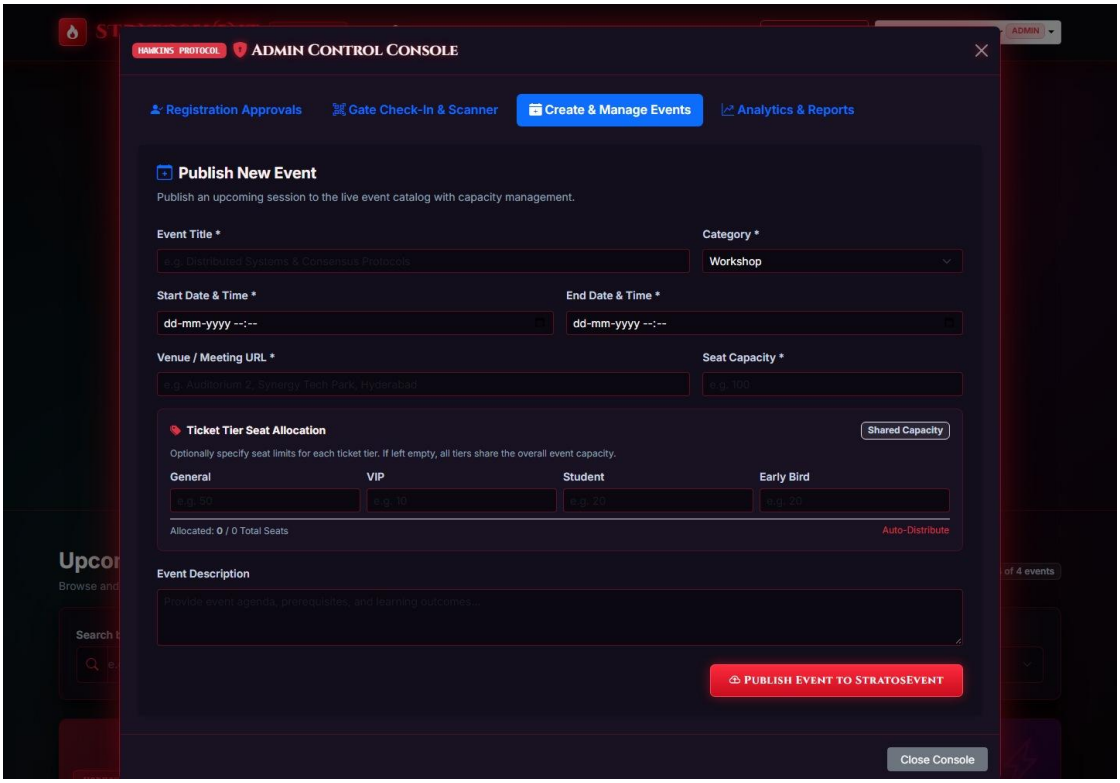


Figure 8: Event creation — schedule, venue, capacity, and per-tier seat allocation

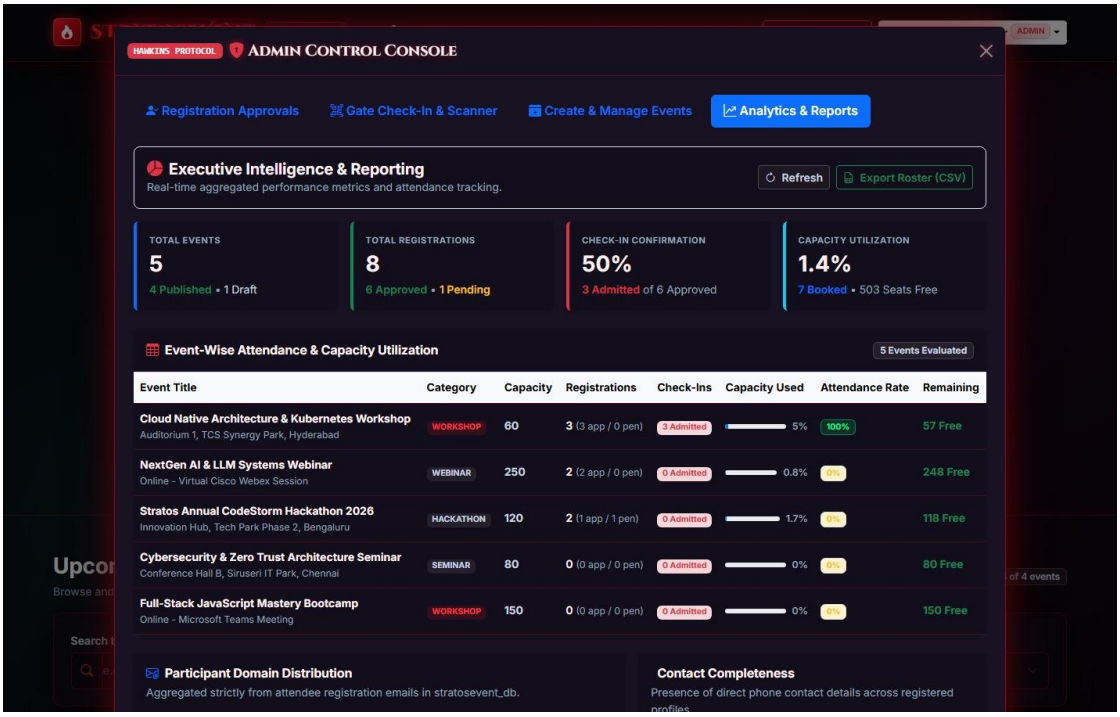


Figure 9: Executive analytics — live KPIs, capacity utilisation, attendance rates, and CSV export

7. Approach / Methodology — Tools and Technologies Used

7.1 Development Methodology

The project followed a milestone-driven, iterative model aligned with the TCS iON programme timeline, with an estimated total effort of 120 hours. Milestone 1 focused on requirements analysis, relational schema design, and the complete frontend: page structure, the Hawkins design system, responsive behaviour, modals, validation feedback, and the event catalogue. Milestone 2 focused on the Express API, MySQL integration, transactional booking logic, role-based middleware, administrator workflows, analytics, automated testing, and documentation.

Each milestone ended with a working, demonstrable increment rather than a document-only review. The frontend was built against a clearly defined API contract so that the two tiers could be developed and tested independently, then integrated without redesign. Automated tests were written alongside the backend routes, and the end-to-end suite was used as the final integration gate before documentation was finalised.

7.2 Tools and Technologies

Technology	Version / Standard	Purpose
Node.js	18+ (tested on v24.20.0)	Server-side JavaScript runtime for the REST API
Express.js	v4.19	HTTP routing, middleware pipeline, error handling
MySQL	8.0+ (InnoDB)	Relational storage, ACID transactions, row-level locking
mysql2	v3.9	Promise-based MySQL driver and connection pool
bcryptjs	v3.0	Salted password hashing (cost factor 10)
jsonwebtoken	v9.0	JWT generation and verification with HS256 pinning
cors	v2.8	Cross-origin resource sharing for the decoupled client
dotenv	v16.4	Environment-based configuration and secret isolation
HTML5 / CSS3	Current standards	Semantic structure, responsive layout, CRT theme, animations
JavaScript (ES6+)	ES2022	Client state, API communication, DOM rendering, filtering
Bootstrap 5	v5.3	Responsive grid, modals, tabs, forms, and utility classes
Bootstrap Icons	v1.11	Interface iconography and status indicators
Nodemon	v3.1 (development)	Automatic server restart during backend development
Postman	Current	API endpoint testing, authentication flows, error verification
Visual Studio Code	Current	Primary editor, integrated terminal, launch and task configuration
Git / GitHub	Current	Version control, history, and source hosting

7.3 Development Workflow

1. Requirements and data modelling — defined the stakeholder roles, functional requirements FR-01 to FR-07, non-functional requirements NFR-01 to NFR-04, acceptance criteria, entity relationships, and the four-table schema with its constraints and indexes.
2. Database implementation — created the schema in MySQL 8.0 InnoDB, applied capacity and date check constraints, composite unique keys, foreign keys with cascade rules, and the tier migration scripts; then loaded the seven-user, five-event, seven-registration, three-attendance seed baseline.
3. Backend development — implemented authentication, event management, registration and ticketing, attendance, and analytics routes; added JWT authentication and role-authorisation middleware, request validation, centralised error responses, and transactional booking logic.
4. Frontend development and integration — built the Hawkins design system, event catalogue, search and category filtering, authentication modal, event detail and tier selector, ticket wallet, and the four-tab administrator console; connected each view to the live API with token-aware requests and graceful error states.
5. Verification and documentation — executed the 35-test backend suite and the 26-step end-to-end suite, audited the database baseline before and after testing, validated responsive layouts, and completed the SRS, traceability matrix, API reference, wireframes, flow diagrams, test log, deployment checklist, technical manual, and this report.

8. System Architecture

StratosEvent uses a decoupled three-tier architecture. The presentation tier is a static browser application; the application tier is an Express API; the data tier is MySQL. The tiers communicate only through a documented REST contract, which allows each to be developed, tested, and deployed independently.

8.1 User Role Architecture

Role	Permissions
Public Visitor	Browse published events, search and filter the catalogue, view event details and live availability, open the authentication modal. No account required.
Participant	All public permissions plus registration, login, tiered pass booking, personal ticket wallet, self-service cancellation, and profile retrieval.
Administrator	Full platform control: create, update, publish, complete, or cancel events; inspect rosters; approve or reject registrations; verify and record gate check-ins; view analytics; export CSV reports.

8.2 Application Flow

1. On first load, the client fetches the published event catalogue from GET /api/events and renders cards with computed available seats.
2. A participant registers or logs in through POST /api/auth/register or POST /api/auth/login. The server verifies the bcrypt hash and returns a signed HS256 JWT containing the user id, email, and role.

3. The client stores the token and attaches it as Authorization: Bearer <token> on every protected request. Express middleware verifies the signature and algorithm before the route handler executes.
4. Event creation and lifecycle changes are restricted to administrator tokens; participant tokens receive HTTP 403, and missing or invalid tokens receive HTTP 401.
5. A booking request enters POST /api/registrations, where a transaction locks the event row, checks capacity and duplicate status, inserts the registration with a unique ticket code, computes the seat number, and commits.
6. Attendance verification reads the pass by ticket code, confirms that its registration is approved and not already admitted, then records the check-in with a timestamp, status, gate notes, and the verifying administrator.
7. Administrator analytics aggregate event, registration, attendance, capacity, and demographic data directly from MySQL and return computed KPIs; the CSV endpoint streams the authorised roster as an attachment.

8.3 Technology Layers

Layer	Technology	Responsibility
Presentation	HTML5, CSS3, ES6+, Bootstrap 5	Interface rendering, responsive layout, client-side filtering, modal workflows, feedback states
Application	Node.js, Express.js, JWT middleware	REST routing, validation, business rules, role enforcement, transactional orchestration
Data	MySQL 8.0 (InnoDB), mysql2 pool	Persistent storage, referential integrity, unique constraints, row locks, aggregation queries
Security	bcryptjs, jsonwebtoken (HS256)	Password hashing, signed stateless sessions, algorithm pinning, production secret guard
Communication	HTTP, JSON, CORS	Client–server data exchange and controlled cross-origin access

8.4 Security Architecture

1. Privilege escalation prevention (CWE-269): public registration ignores any client-supplied role and unconditionally assigns participant; administrative accounts cannot be created through the public endpoint.
2. Credential protection: passwords are hashed with bcrypt at cost factor 10 and capped at 128 characters to avoid CPU-exhaustion attacks; plaintext passwords are never stored or logged.
3. Token hardening: JWTs are created and verified with an explicit HS256 algorithm allow-list, preventing algorithm confusion and downgrade attempts.
4. Production guard: the server refuses to start in production when JWT_SECRET is missing or still set to a documented default placeholder.
5. Injection defence: every SQL statement uses parameter placeholders through mysql2; user input is validated and escaped before rendering.

9. Database Design

The database is MySQL 8.0 with InnoDB, accessed through a promise-based connection pool. Four tables model the complete lifecycle: users own registrations, events contain registrations, and attendance records validate individual registrations. Referential integrity is enforced by foreign keys rather than application convention.

9.1 Schema Overview

Table	Primary Key	Description
users	id (INT UNSIGNED, AUTO_INCREMENT)	Stores full name, unique email, bcrypt password hash, role (admin or participant), optional phone number, and timestamps. Seeded with two administrators and five participants.
events	id (INT UNSIGNED, AUTO_INCREMENT)	Stores title, description, event type, start and end time, venue or meeting link, total capacity, per-tier capacity columns, lifecycle status, creator reference, and timestamps.
registrations	id (INT UNSIGNED, AUTO_INCREMENT)	Stores the participant, event, unique ticket code, ticket tier, lifecycle status (pending, approved, rejected, cancelled), and registration date.
attendance	id (INT UNSIGNED, AUTO_INCREMENT)	Stores the registration being admitted, check-in timestamp, status (checked_in, late, absent), verifying administrator, and gate notes.

9.2 Relationships and Constraints

- users 1 : N registrations — a participant may hold many passes; deleting a user cascades to their registrations.
- events 1 : N registrations — an event contains many bookings; deleting an event cascades to its registrations and their attendance records.
- registrations 1 : 0..1 attendance — a pass may be checked in at most once; the unique constraint uq_attendance_registration enforces this at the database level.
- uq_registrations_user_event (user_id, event_id) makes duplicate booking impossible even if two requests arrive simultaneously.
- uq_registrations_ticket_code and uq_users_email protect ticket identity and account identity respectively.
- chk_events_capacity (capacity > 0) and chk_events_dates (end_time >= start_time) prevent invalid events at write time.
- Foreign keys use ON DELETE CASCADE for dependent records and ON DELETE SET NULL for the optional event creator and verifying administrator.

9.3 Ticket Tier Extension

Two non-destructive migrations extend the core schema for tiered passes. Migration 001 adds a ticket_tier enumeration (general, vip, student, early_bird) to registrations with a default of general and an index for filtering. Migration 002 adds optional capacity_general, capacity_vip, capacity_student, and capacity_early_bird columns to events. When these columns are null, all tiers share the event's overall capacity; when they are set, each tier has an independent quota, and the booking transaction validates availability against both the tier limit and the total venue limit.

9.4 Sample Data

The seed script establishes a realistic baseline without destructive truncation: two administrator accounts, five participant accounts, five events spanning workshop, webinar, hackathon, and seminar categories, seven registrations in approved and pending states, and three attendance records covering checked_in and late outcomes. Four of the five events are published; the fifth remains in draft to demonstrate lifecycle control. At the time of documentation, verification activity had added one further registration, so the live analytics view in Figure 9 reports eight registration records.

9.5 Indexing Strategy

- idx_users_role supports role-filtered administrative queries.
- idx_events_status, idx_events_start_time, and idx_events_type accelerate the public catalogue and its filters.
- idx_registrations_user, idx_registrations_event, idx_registrations_status, and idx_registrations_ticket_tier support participant wallets, event rosters, moderation queues, and tier filtering.
- idx_attendance_time and idx_attendance_status support attendance reporting and check-in audits.

10. Module-wise System Description

10.1 Authentication and Authorisation Module

Registration accepts a full name, unique email, password, and optional phone number. The password is hashed with bcrypt before insertion, and the assigned role is always participant regardless of the request payload. Login compares the submitted password with the stored hash and, on success, returns a signed JWT with a configurable expiry along with the public user profile. The authentication middleware validates the Authorization header, verifies the HS256 signature, attaches the decoded identity to the request, and rejects missing or malformed tokens with HTTP 401. The authorisation middleware then checks the required role and returns HTTP 403 for authenticated users who lack permission.

10.2 Event Discovery Module

GET /api/events returns published events with a computed registered count and available-seat value derived from active registrations, so availability is never a stale manually maintained number. The endpoint supports status, event_type, and search query parameters. The client applies instant keyword and category filtering for responsiveness while the server remains the source of truth. Each

card displays category, title, description, schedule, venue, capacity, and a visual availability indicator; sold-out events disable the registration action.

10.3 Registration and Booking Module

POST /api/registrations is the critical path of the platform. It requires an authenticated participant, accepts an event id and an optional ticket tier, and runs inside a dedicated MySQL connection with an explicit transaction. The event row is locked with SELECT ... FOR UPDATE, active registrations are counted within the same transaction, tier and total capacity are validated, duplicate status is checked, and the registration is inserted with a generated ticket code. The seat number is derived from the count of active registrations. Any failure rolls the transaction back; success commits and returns the confirmed pass. This design guarantees that two simultaneous requests for the final seat cannot both succeed.

10.4 Participant Ticket Wallet and Cancellation

GET /api/registrations/my returns the authenticated participant's passes with event details, ticket tier, ticket code, status, and check-in state. The wallet groups the information into a compact table and exposes a cancellation action for eligible bookings. PATCH /api/registrations/:id/cancel verifies ownership, marks the registration cancelled, and releases the seat. Because availability is computed from active registrations, the released seat reappears in the catalogue immediately without a separate counter update.

10.5 Administrator Moderation Module

Administrators can list registrations by event and tier, approve or reject pending passes, and inspect participant identity and contact details. Every administrative route is protected by both authentication and role middleware; the UI also hides administrative controls from non-admin users, but the server check is the authoritative control. Event management supports creation, update, lifecycle transition, and safe deletion: an event with registrations is archived to cancelled, while an empty event may be hard-deleted.

10.6 Attendance and Gate Check-In Module

The gate workflow has two steps. GET /api/attendance/verify/:ticketCode performs a non-destructive inspection: it returns the attendee, event, tier, registration status, and whether the pass has already been admitted. If the pass is valid, the administrator records admission through POST /api/attendance/check-in with a status and optional gate notes. The unique constraint on registration_id guarantees that a pass cannot be admitted twice; a second attempt returns HTTP 409 with a duplicate check-in error. The event roster endpoint aggregates registered, approved, checked-in, and attendance-percentage figures.

10.7 Analytics and Reporting Module

The analytics endpoints aggregate directly from MySQL rather than from cached client state. The overview returns event counts by status and type, registration counts by status, checked-in totals, the attendance confirmation ratio, published capacity, active reservations, utilisation percentage, remaining seats, and demographic aggregates based on email domains and phone-number presence. The event-level endpoint returns capacity, registrations, approvals, pending count, check-ins, utilisation, attendance rate, and remaining seats for every event. The CSV endpoint streams a

complete, authorised roster with registration, event, participant, and attendance columns for audit and offline analysis.

11. Booking and Ticketing Logic

11.1 Booking Flow

1. The participant browses the published catalogue, searches or filters by category, and opens an event detail modal.
2. The participant selects an available ticket tier; the interface displays the remaining seats for that tier and disables unavailable options.
3. If no session exists, the client opens the authentication modal and returns the participant to the same event after login.
4. The client sends POST /api/registrations with the event id, selected tier, and the JWT in the Authorization header.
5. The authentication middleware verifies the token and supplies the participant identity; the route opens a transaction and locks the event row with SELECT ... FOR UPDATE.
6. Within the transaction, the server counts active registrations, validates total and tier capacity, checks for an existing registration, and generates a unique ticket code.
7. The registration is inserted with approved status and a computed seat number; the transaction commits and the confirmed pass is returned.
8. The client displays the digital pass and adds it to the participant's ticket wallet. If any validation fails, the transaction rolls back and a descriptive 409 or 400 response is shown.

11.2 Ticket Code Generation

Live tickets use the format SE-<eventId>-<userId>-<timestamp36>-<hex4>, for example SE-1-3-M8A29F-4B9F1A. The timestamp component is Base36-encoded to keep the code compact, and the final component adds cryptographic randomness. The registrations table declares ticket_code as UNIQUE, so even a theoretical collision is rejected by the database. Seeded demonstration records use the readable form TKT-STR-2026-0001 through TKT-STR-2026-0007, which makes manual testing and gate demonstrations straightforward without weakening the production format.

11.3 Concurrency and Seat Integrity

The essential guarantee is that availability is never read outside the lock that protects the subsequent insert. The booking transaction executes SELECT ... FOR UPDATE on the event row, which serialises competing bookings for that event. Each transaction then sees the committed state left by the previous one, counts the active registrations, and decides whether capacity remains. The composite unique key on (user_id, event_id) prevents duplicate bookings independently of the capacity check, and the ticket-code unique key prevents duplicate pass identities. This combination is what allows the API to return a deterministic 409 Conflict for duplicate or capacity conditions instead of silently overselling.

11.4 Cancellation and Seat Release

Cancellation does not delete the registration. It changes the status to cancelled after verifying that the requester owns the pass or is an administrator. Since capacity is computed from registrations that are not cancelled, the seat becomes available to the next booking immediately. Keeping the cancelled row preserves the audit trail and allows analytics to report cancellation volume accurately.

11.5 Tier Allocation

When an administrator configures tier capacities, the event form enforces that the allocated tier seats do not exceed total capacity. The form offers an auto-distribute action for even allocation and a live allocated-versus-capacity indicator. During booking, the server checks both the selected tier's remaining quota and the event's overall remaining capacity, so a tier can sell out independently while the venue still has seats in other categories. If no tier capacities are configured, all tiers draw from the shared venue pool.

12. User Interface Design

The interface uses a custom Hawkins Laboratory design system layered over Bootstrap 5. The system is implemented in a single stylesheet with reusable tokens, so colour, spacing, typography, borders, and glow behaviour remain consistent across public and administrative surfaces.

12.1 Design Language

- Colour palette: void black (#0a0709) for the laboratory background, Hawkins crimson (#ff1744) for primary actions and danger states, upside-down teal (#00e5ff) for informational accents, amber (#ff9100) for warnings, and off-white for body text.
- Typography: a retro-futuristic serif display face for headings, paired with monospace instrumentation for codes, metrics, status readouts, and technical labels.
- Surface treatment: dark panels, fine grid lines, CRT scanline overlays, restrained red and teal glow, and high-contrast borders that separate administrative controls from attendee content.
- Motion: a pulsing live indicator, hover elevation on event cards, modal transitions, and loading spinners that communicate system activity without distracting from the task.
- State legibility: colour-coded status pills distinguish published, draft, pending, approved, rejected, cancelled, checked-in, late, and sold-out states.
- Responsive typography and spacing: fluid heading sizes, stacked toolbars, and touch targets sized for mobile use.

12.2 Page-by-Page Design Summary

Screen	Key Design Elements	Figure
Landing / Discovery	Hawkins hero, live pill, primary calls to action, value strip, event catalogue, search and category toolbar	Figures 1–2
Authentication	Focused modal with Sign In and Create Account tabs, demo account access, inline validation alerts	Figure 3
Event Detail	Modal with schedule, venue, capacity, tier selector, live per-tier availability, register action	Figure 4

Screen	Key Design Elements	Figure
Participant Wallet	Pass table with ticket code, tier, status, event context, and cancellation controls	Figure 5
Admin Console	Four-tab operational surface: approvals, gate scanner, event creation, analytics and CSV export	Figures 6–9
Mobile Experience	Collapsed navigation, stacked hero, single-column cards, full-width controls, zero horizontal overflow	Figures 10–11

12.3 Responsive Design

Responsive behaviour was treated as a functional requirement rather than a finishing step. The layout was validated at desktop (1400 px), tablet (768 px), and mobile (375–390 px) widths. At mobile width the navigation collapses, the hero stacks vertically, the discovery toolbar becomes a single column, event cards occupy the full content width, and modal content scrolls internally. Document overflow is measured at zero pixels horizontally, so no control is pushed off-screen.

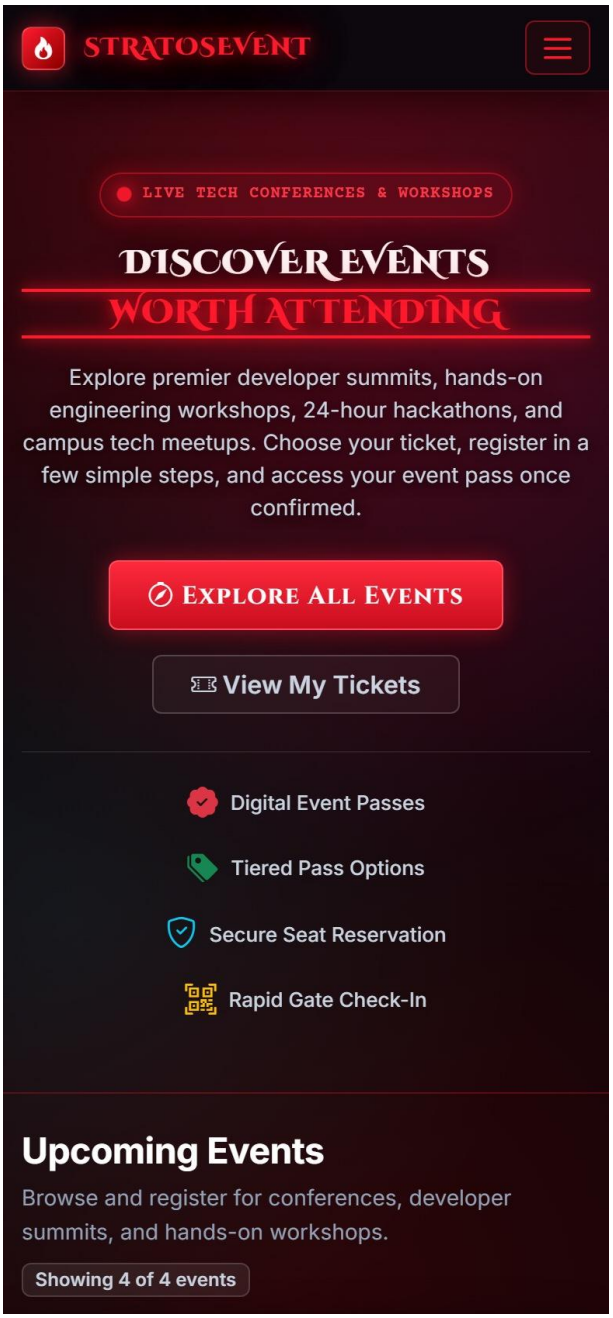


Figure 10: Mobile landing page — collapsed navigation, stacked hero, and full-width calls to action

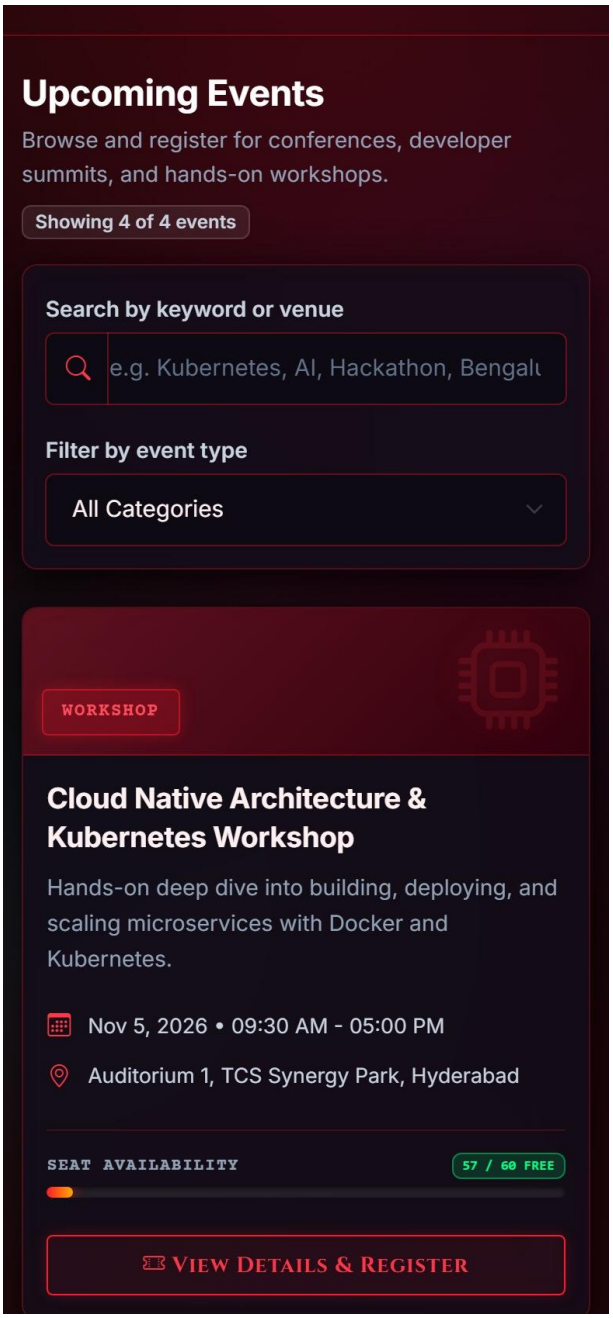


Figure 11: Mobile event catalogue — single-column cards and touch-friendly controls with zero horizontal overflow

12.4 Feedback and Accessibility

Every asynchronous action has a visible state: buttons enter a disabled loading state, empty results show a descriptive message, API failures render an alert with the server's error message, and destructive or irreversible actions ask for confirmation. Form controls carry labels, modal titles are associated with their dialogs, status changes use text as well as colour, and interactive elements retain visible focus states. The interface therefore remains understandable for keyboard and assistive-technology users while preserving the theme.

13. Key Features

Feature	Description
Three-tier access control	Public discovery, authenticated participant workflows, and administrator operations, enforced on the server and reflected in the interface.
JWT authentication	Stateless HS256-signed sessions with explicit algorithm pinning and token verification on every protected route.
Bcrypt password security	Cost-factor-10 salted hashing; plaintext credentials are never stored or logged.
Privilege-escalation defence	Public registration discards client-supplied roles and always creates a participant (CWE-269 mitigation).
Live event catalogue	Published events with computed seat availability, category badges, schedules, venues, and detailed modals.
Search and category filters	Instant keyword matching across title, description, and venue, combined with workshop, webinar, conference, hackathon, and seminar filters.
Tiered passes	General, VIP, Student, and Early Bird allocations with optional per-event quotas and live per-tier availability.
Concurrency-safe booking	Transactional SELECT ... FOR UPDATE serialises competing reservations and prevents overselling.
Duplicate protection	Composite unique key on user and event, plus unique ticket codes and unique attendance registration keys.
Digital ticket wallet	Participants can view every pass, inspect its status and tier, and cancel eligible bookings to release the seat.
Administrator moderation	Event-scoped registration rosters with tier filtering and approve or reject actions.
Gate check-in scanner	Pass inspection before admission, timestamped check-in, gate notes, status recording, and duplicate scan rejection.
Executive analytics	Live KPI cards, event-wise capacity and attendance performance, participant domain distribution, and CSV roster export.
Responsive Hawkins UI	A distinctive, accessible dark-laboratory theme with zero horizontal overflow at mobile width.
Automated verification	35 backend tests and 26 end-to-end flows with database-state preservation and 100% pass rates.

14. Admin Panel

The administrator console is the operational centre of StratosEvent. It is accessible only to accounts with the admin role, and the restriction is enforced twice: the interface does not render administrative actions for participants, and every administrative endpoint independently verifies the role from the verified token. A participant who crafts a direct request therefore receives HTTP 403 rather than access.

The console is organised into four tabs so that high-frequency gate operations are never buried under configuration screens: Registration Approvals, Gate Check-In & Scanner, Create & Manage Events, and Analytics & Reports. Figure 6 shows the approvals roster; Figure 7 shows a successful gate verification; Figure 8 shows event creation with tier allocation; Figure 9 shows the analytics surface.

Capability	Description	API Endpoint
Registration roster	View all registrations for a selected event, filtered by ticket tier, with attendee identity, contact details, ticket code, tier, date, and status.	GET /api/registrations?event_id=…
Approve registration	Move a pending registration to approved, making the pass eligible for admission.	PATCH /api/registrations/:id/status
Reject registration	Reject a registration that does not meet event criteria, removing it from active capacity.	PATCH /api/registrations/:id/status
Verify pass	Inspect a ticket code and return attendee, event, tier, status, and prior check-in state without modifying data.	GET /api/attendance/verify/:ticketCode
Record check-in	Admit a valid pass, record the timestamp, status, verifying administrator, and gate notes.	POST /api/attendance/check-in
Create event	Publish an event with title, category, schedule, venue, capacity, description, and per-tier allocation.	POST /api/events
Lifecycle management	Update event details or transition status between draft, published, completed, and cancelled.	PUT /api/events/:id; PATCH /api/events/:id/status
Analytics overview	View platform KPIs, registration breakdown, attendance ratio, capacity utilisation, and demographics.	GET /api/analytics/overview
Event performance	Compare per-event capacity, registrations, approvals, check-ins, utilisation, attendance rate, and remaining seats.	GET /api/analytics/events
CSV export	Download a complete registration and attendance roster as an authorised CSV attachment.	GET /api/analytics/export/csv

15. Challenges and Opportunities

15.1 Challenges Encountered

- Concurrency correctness: the first booking implementation read availability and then updated it in separate statements. Two simultaneous requests could both observe a free seat. The solution was to move the read, capacity check, insert, and seat computation inside one transaction with SELECT ... FOR UPDATE, and to add the composite unique constraint as a second line of defence.

- Tier capacity arithmetic: supporting four independent tier quotas while still respecting total venue capacity required careful validation at both event creation and booking time. Partial allocations, sold-out tiers, and shared-pool events all had to produce predictable behaviour without fractional or negative availability.
- Authentication boundaries: distinguishing 401 (no valid identity) from 403 (valid identity, insufficient role) required a consistent middleware order and uniform error envelopes across every route, including routes that nest resources such as registrations and attendance.
- Privilege-escalation testing: proving that a role injected into a registration payload is ignored required an explicit negative test that then attempts to use the issued token against an administrative endpoint.
- Frontend state without a framework: managing authentication state, catalogue state, modal instances, loading states, filters, and the admin roster without a client framework demanded disciplined rendering functions and explicit event binding rather than implicit reactivity.
- Visual complexity with performance constraints: scanlines, glow, layered backgrounds, and animated indicators can easily cost paint time or cause mobile overflow. The theme was therefore built with reusable tokens, transform-based animation, and repeated overflow measurements at 375–390 px.
- Test isolation: running destructive-looking verification flows against a seeded database without leaving residue required setup and teardown that create their own users and events, track the created identifiers, and remove them in dependency order.

15.2 Opportunities Identified

- The API is already client-agnostic, so a React Native or Flutter mobile client could consume the same endpoints without backend changes.
- The verified ticket code is a natural payload for a camera-based QR scanner, enabling contactless gate admission and reducing manual entry.
- Ticket tiers create a direct path to paid passes through Razorpay or Stripe, with pricing, refunds, and settlement reporting.
- The analytics layer can be extended into scheduled reports by email, trend charts, and export formats beyond CSV.
- The event lifecycle model supports waitlists, invitations, and multi-session events without changing the core booking transaction.
- Multi-tenant organisation accounts would allow the same platform to serve several colleges or business units with isolated catalogues and administrators.

16. Advantages of the System

- Correctness under concurrency: row-level locking and database constraints make overselling and duplicate booking structurally difficult, not merely unlikely.
- Layered security: bcrypt, HS256-pinned JWTs, server-side role checks, parameterised SQL, and the CWE-269 registration guard cover the most common credential and authorisation failures.
- Single source of truth: availability, attendance, and analytics are computed from the database, so the interface cannot drift from reality.

- Complete lifecycle coverage: the platform handles discovery, booking, moderation, admission, cancellation, and reporting in one coherent system.
- Auditability: registrations are status-transitioned rather than deleted, attendance is timestamped and attributed, and CSV export preserves a portable record.
- Low operational weight: a static frontend and a single Node service avoid a build pipeline, a client dependency tree, and a complex deployment topology.
- Deliberate user experience: the Hawkins theme creates clear visual hierarchy and immediate state recognition while remaining responsive and keyboard-accessible.
- Verification evidence: two independent automated suites, 61 total checks, and a preserved database baseline provide concrete quality evidence rather than claims.

17. Limitations

- No payment gateway: passes are free; tier names are operational categories rather than priced products.
- No automated notifications: confirmations, approvals, rejections, and reminders are not sent by email or SMS.
- Manual ticket entry at the gate: the scanner verifies typed or pasted codes but does not decode QR or barcode images from a camera.
- Token storage: the client keeps the JWT in localStorage, which is convenient but exposed to cross-site scripting; HTTP-only cookies would be safer in production.
- Single-instance deployment: the transaction design is correct for a single MySQL server, but horizontal scaling would require connection-pool tuning and infrastructure-level coordination.
- Local environment orientation: there is no container image, reverse proxy, TLS termination, or managed database configuration in the delivered scope.
- Analytics depth: the reporting engine covers operational KPIs and rosters but not predictive demand, cohort retention, or financial reconciliation.
- No waitlist or invitation workflow: full events reject new bookings rather than queueing interested participants.
- No internationalisation or time-zone conversion: timestamps are handled in the server's configured local context.
- No native mobile application or offline mode; mobile access is through the responsive web interface.

18. Testing Documentation

Verification combined a 35-test automated backend suite, a 26-step end-to-end operational suite, manual interface checks, and database-state audits. The backend suite exercises authentication, authorisation, event management, booking concurrency, cancellation, attendance, and analytics. The end-to-end suite follows real user journeys from catalogue discovery through gate admission and reporting. Both suites completed with a 100% pass rate.

18.1 Functional Test Case Summary

TC #	Test Case	Pre-requisites	Expected Result	Status
TC01	Participant registration	Unique email; server running	201 Created; participant record stored; JWT returned	Pass
TC02	Duplicate email registration	Existing account	409 Conflict with a descriptive error	Pass
TC03	Privilege escalation attempt	Public registration endpoint	Role payload ignored; account created as participant; admin route returns 403	Pass
TC04	Login — valid and invalid credentials	Seeded participant account	200 with token on valid password; 401 on invalid password	Pass
TC05	Protected route without token	No Authorization header	401 Unauthorized	Pass
TC06	Participant calls admin endpoint	Participant JWT	403 Forbidden; no administrative data returned	Pass
TC07	Event creation and update	Administrator JWT	201 Created and 200 OK with persisted changes	Pass
TC08	Catalogue search and category filter	Published events in database	Matching events returned; draft events excluded	Pass
TC09	Successful booking	Participant JWT; capacity available	201 Created; unique ticket code; seat number computed	Pass
TC10	Duplicate booking	Participant already registered	409 Conflict (DUPLICATE_REGISTRATION)	Pass
TC11	Capacity guard	Event filled to capacity	409 Conflict (CAPACITY_REACHED); transaction rolled back	Pass
TC12	Concurrent final-seat booking	One seat remaining; two requests	Exactly one succeeds; one fails; capacity never exceeded	Pass
TC13	Self-service cancellation	Active approved booking	200 OK; status cancelled; seat released to availability	Pass
TC14	Gate verification and check-in	Approved, unchecked pass	200 verification; 201 check-in with timestamp and status	Pass
TC15	Duplicate check-in	Pass already admitted	409 Conflict (DUPLICATE_CHECKIN)	Pass
TC16	Analytics and CSV export	Administrator JWT	Computed KPIs returned; valid text/csv attachment streamed	Pass

18.2 Test Scenario Template

Req ID	Scenario ID	Application / Screen	High-Level Test Conditions	Priority
REQ01	TS01	Authentication	Register with valid and invalid data; login with valid and invalid credentials	High
REQ02	TS02	Role Security	Attempt administrative actions with no token, a participant token, and a tampered payload	High
REQ03	TS03	Event Management	Create, update, publish, complete, and cancel events; verify public visibility rules	High
REQ04	TS04	Booking Engine	Book with available capacity, duplicate booking, filled capacity, and simultaneous final-seat requests	High
REQ05	TS05	Attendance	Verify a valid pass, admit it, re-scan it, and confirm the roster and attendance metrics	High
REQ06	TS06	Analytics	Compare overview and event-level metrics with database records; download and inspect CSV export	Medium
REQ07	TS07	Responsive UI	Validate layout, navigation, modals, and controls at 1400, 768, and 390 px; measure horizontal overflow	Medium

18.3 Automated Backend Suite

The backend suite is implemented with the Node.js built-in test runner and executes 35 assertions across nine functional groups: system health, registration and privilege defence, authentication and token validation, event management, registration and concurrency, cancellation, administration and pass issuance, attendance, and analytics with CSV export. The recorded execution completed in approximately 1.5 seconds with 35 passed and 0 failed.

18.4 End-to-End Verification and Data Preservation

The end-to-end suite performs 26 sequential operational checks that mirror a real event cycle: health check, catalogue queries, administrator login, event creation, tier allocation, participant registration and login, booking, duplicate and capacity rejection, roster inspection, approval, pass verification, check-in, duplicate check-in rejection, analytics comparison, CSV export, cancellation, and cleanup. Before and after the run, the database was audited against the seeded baseline of seven users, five events, seven registrations, and three attendance records. The baseline was preserved in full, confirming that the test harness cleans up only the records it creates and never damages existing data.

19. Reflections on the Project

Building StratosEvent end to end made the distance between knowing individual technologies and integrating them into a reliable product very concrete. Writing an Express route is straightforward; designing a booking route that remains correct when two users submit the final seat at the same

moment requires understanding transactions, isolation, lock scope, constraint behaviour, and error recovery as one system. That was the most valuable technical lesson of the project.

The decision to implement genuine authentication rather than a simulated login flag raised the complexity substantially. It also forced a clear separation between identity (who the user is) and authorisation (what the user may do), which in turn made the privilege-escalation test meaningful. Seeing a crafted role payload produce a participant account, and then seeing that account receive 403 on an administrative route, was a more convincing demonstration of security than any diagram could provide.

The analytics work changed how I thought about the database. Computing availability, attendance ratios, utilisation, and demographics directly from relational data made it obvious that reporting is not a separate feature bolted onto an application; it is a consequence of modelling the lifecycle correctly. Because registrations are status-transitioned and attendance is uniquely bound to a registration, the reports are trustworthy without a separate reporting store.

The interface was equally instructive. A distinctive visual identity is not incompatible with usability, but it requires discipline: consistent spacing tokens, deliberate contrast, responsive type, and repeated overflow testing. The Hawkins theme gives the platform a memorable character, yet the interaction patterns remain conventional — search, filter, open, select, confirm — so the theme never becomes a barrier to completing the task.

Finally, the testing discipline changed the quality of the deliverable. Writing negative tests for duplicate booking, capacity exhaustion, role escalation, and duplicate check-in exposed assumptions that would otherwise have remained hidden. The preserved database baseline after 61 checks is, in practical terms, the strongest evidence that the system behaves as documented.

20. Recommendations

- Store JWTs in HTTP-only, Secure, SameSite cookies instead of localStorage to reduce exposure to cross-site scripting, and add refresh-token rotation with server-side revocation.
- Add rate limiting and account lockout on authentication endpoints, together with a content security policy and strict security headers.
- Introduce a payment gateway such as Razorpay for INR transactions or Stripe for international payments, with idempotent payment webhooks and refund handling.
- Add transactional email through Nodemailer or a transactional provider for booking confirmation, approval or rejection, event reminders, and cancellation receipts.
- Implement camera-based QR decoding in the gate scanner, with an offline fallback queue for network interruptions during admission.
- Add a waitlist and automatic seat-offer flow so that cancelled seats are offered to interested participants before returning to the public pool.
- Containerise the application with Docker Compose, add a reverse proxy with TLS, and define managed database, backup, and restore procedures before production use.
- Extend analytics with time-series booking trends, conversion funnels, no-show prediction, and scheduled PDF or CSV reports for organisers.
- Add calendar export (ICS) and optional Google or Outlook calendar synchronisation for confirmed passes.

- Introduce multi-tenant organisation accounts, audit logging for administrative actions, and configurable event approval policies.

21. Outcome / Conclusion

StratosEvent successfully delivers a complete, working, full-stack event management and registration platform that meets the objectives defined at the start of the project. Public visitors can discover and evaluate events; participants can register, reserve tiered passes, view their ticket wallet, and cancel bookings; administrators can publish events, moderate registrations, verify passes at the gate, and analyse performance. Every one of those actions is backed by an authorised API endpoint and a persistent database operation.

The engineering value of the project lies in the guarantees rather than the feature list. Capacity is protected by row-level locking inside an atomic transaction. Duplicate bookings and duplicate check-ins are blocked by database constraints. Privilege escalation is blocked by unconditional server-side role assignment. Passwords are hashed, tokens are algorithm-pinned, SQL is parameterised, and the production startup path refuses insecure secrets. Those guarantees are what make the interface trustworthy.

The verification results support that claim: 35 backend tests and 26 end-to-end flows passed without failure, the responsive layout produced zero horizontal overflow at mobile width, and the seeded database baseline remained intact after the full run. The screenshots in this report show the system operating against live data, not staged mock-ups.

The project therefore meets its academic requirement and, more importantly, demonstrates the standard of thinking expected in professional full-stack work: model the domain correctly, enforce integrity at the lowest reliable layer, expose only the operations each role needs, and make the resulting state visible and understandable to the people using it.

Source code: https://github.com/abhinavsankar27-bot/StratosEvent

22. Enhancement Scope

- Payment integration: Razorpay for INR ticket sales and Stripe for international payments, including sandbox testing, webhook verification, refunds, and financial reconciliation.
- Notification service: transactional email and SMS for registration confirmation, approval or rejection, reminders, cancellation, and gate check-in receipts.
- Camera QR scanning: an in-browser decoder for gate admission, with offline caching and deferred synchronisation when connectivity is unavailable.
- Real-time availability: a WebSocket or server-sent events channel that pushes seat changes to every participant viewing an event, reducing failed booking attempts.
- Waitlist automation: queue interested participants when an event is full and offer released seats in order with an expiry window.
- Organiser analytics: interactive charts for registration velocity, tier mix, attendance conversion, and cancellation trends, with scheduled report delivery.
- Cloud deployment: Docker images, a managed MySQL instance, object storage for assets, TLS termination, centralised logging, and automated backups.

- Multi-tenant administration: organisation-scoped catalogues, administrators, branding, and approval policies with strict data isolation.
- Accessibility and internationalisation: a formal WCAG 2.1 AA audit, screen-reader testing, localisation, and time-zone-aware scheduling.
- Mobile client: a React Native or Flutter application consuming the existing REST API, with digital wallet integration for passes.

23. Link to Code and Executable File

The complete source code — including the Express REST API, the responsive Hawkins Laboratory frontend, the MySQL schema and seed scripts, the tier migrations, the automated test suites, and all project documentation — is maintained in a Git repository and published on GitHub:

GitHub Repository: https://github.com/abhinavsankar27-bot/StratosEvent

Local project path: C:\Users\Admin\OneDrive\Desktop\StratosEvent

To run the application locally, install Node.js 18 or later and MySQL 8.0 or later. Create the database by executing database/schema.sql, apply the migrations in database/migrations in numeric order, and load the demonstration baseline with database/seed.sql. Copy backend/.env.example to backend/.env, set the MySQL credentials, database name, port, and a strong JWT secret, then run npm install followed by npm start inside the backend directory. Serve the frontend directory with any static file server; the included Visual Studio Code tasks can start both services, run the 35-test backend suite, and execute the 26-step end-to-end verification.

The backend API is available at http://localhost:5000, the health check at http://localhost:5000/api/health, and the frontend at http://localhost:3000 when using the provided task configuration. The eleven screenshots in this report were captured from the fully running local application, confirming successful end-to-end integration of the frontend, API, and database.

24. Research Questions and Responses

Q1: Why was MySQL chosen over MongoDB for this project?

The core domain is inherently relational. A registration is a relationship between one participant and one event; attendance is a relationship between one registration and one admission event. Those relationships need foreign keys, composite uniqueness, and transactional consistency. MySQL 8.0 with InnoDB provides all three, together with row-level locking through SELECT ... FOR UPDATE and check constraints for capacity and date validity. MongoDB is excellent for document-shaped data with variable structure, but using it here would have moved integrity checks into application code, which is precisely where concurrency bugs are hardest to eliminate.

Q2: Why was a vanilla JavaScript and Bootstrap frontend chosen instead of a framework such as React?

The application is forms, tables, modals, and a filtered catalogue; it does not need a virtual DOM, a client-side router, or a complex state graph. Vanilla ES6+ with Bootstrap keeps the first load small, removes the build chain and its dependency vulnerabilities, and makes the source directly inspectable during assessment. The trade-off is that state management must be explicit: the code uses focused render functions, event delegation, and named modal instances rather than implicit reactivity. For the

scale of this interface that trade-off is favourable; for a larger product with many concurrent client states, a framework would become the better choice.

Q3: How is overbooking prevented when two participants attempt to book the final seat simultaneously?

The booking endpoint opens a MySQL transaction and locks the event row with SELECT ... FOR UPDATE. The second request blocks until the first transaction commits or rolls back, then reads the updated registration count inside its own transaction. If capacity is exhausted, it rolls back and returns 409 Conflict (CAPACITY_REACHED). The composite unique key on (user_id, event_id) independently prevents duplicate bookings, and the unique ticket-code constraint prevents duplicate pass identities. The guarantee therefore does not depend on timing or on the interface disabling a button.

Q4: How does the system prevent privilege escalation during public registration?

The registration controller never reads the role from the request body. It assigns the literal participant role before insertion, so a payload containing role: "admin" or role: "administrator" produces an ordinary participant account. The issued token consequently carries participant claims, and administrative routes reject it with 403 Forbidden. This is a direct CWE-269 mitigation: authority is derived from server-side policy, never from client-supplied data. The behaviour is covered by explicit negative tests in the backend suite.

Q5: How does the gate check-in workflow prevent ticket reuse?

A pass is admitted by creating an attendance record whose registration_id is unique. The verification endpoint first inspects the pass without modifying data, returning the attendee, event, tier, registration status, and prior check-in state. If the pass is eligible, the check-in endpoint inserts the attendance row. A second attempt violates uq_attendance_registration, and the API translates the database error into 409 Conflict (DUPLICATE_CHECKIN). Because the rule is enforced by the database rather than by a UI flag, it remains correct even if two administrators scan the same pass at different gates.

Q6: What was the reasoning behind the Hawkins Laboratory visual theme?

Event platforms tend to look interchangeable. The Hawkins theme gives StratosEvent an immediately recognisable identity while serving a functional purpose: dark laboratory surfaces reduce glare in venue and gate environments, crimson and teal create unmistakable action and information hierarchies, and monospace instrumentation makes codes, metrics, and statuses easy to scan. The theme was implemented with reusable CSS tokens and responsive constraints rather than image-heavy artwork, so it remains fast and produces zero horizontal overflow on mobile. The result is a platform that feels distinctive without making the registration task harder.

25. References

- Node.js Documentation — Official runtime, test runner, and API reference. https://nodejs.org/docs
- Express.js Documentation — Express 4.x routing, middleware, and error handling. https://expressjs.com

- MySQL 8.0 Reference Manual — InnoDB transactions, locking, constraints, and SQL syntax. https://dev.mysql.com/doc
- mysql2 npm Package — Promise-based MySQL driver and connection pooling. https://www.npmjs.com/package/mysql2
- bcryptjs npm Package — Password hashing library. https://www.npmjs.com/package/bcryptjs
- jsonwebtoken npm Package — JWT signing, verification, and algorithm options. https://www.npmjs.com/package/jsonwebtoken
- Bootstrap 5 Documentation — Responsive grid, components, utilities, and accessibility. https://getbootstrap.com/docs
- MDN Web Docs — HTML, CSS, JavaScript, Fetch API, and accessibility guidance. https://developer.mozilla.org
- OWASP — Authorization Cheat Sheet and CWE-269: Improper Privilege Management. https://owasp.org
- Postman Learning Center — API testing and collection documentation. https://learning.postman.com
- Git and GitHub Documentation — Version control and repository hosting. https://docs.github.com
- TCS iON Industry Project Programme — Guidelines and submission requirements. https://ionguidedlearning.tcsion.com